

MANUAL DE OPERACIÓN Y CONEXIONES

Carrito Robótico Educativo

Programación y robótica para niños
Proyecto de Practicas Comunitarias — ESPOL

Integrantes: Andry Ortega Vera, Andrea Chávez Mateo

1. Introducción

Este manual es una guía didáctica paso a paso, complementaria a las diapositivas ya entregadas, para garantizar el correcto uso del kit robótico durante las sesiones con los estudiantes. No es un simple instructivo de encendido: detalla cómo se arman los componentes, donde y como se programa el código, y que hacer en cada una de las actividades planteadas.

El flujo que sigue este documento es el siguiente:

1. Conocer los componentes del kit y su función.
2. Conectar el circuito siguiendo el diagrama de conexiones.
3. Instalar y configurar el software necesario (Arduino IDE y Tinkercad).
4. Programar y cargar las instrucciones en el Arduino Nano.
5. Ejecutar el código y observar el comportamiento del carrito.
6. Experimentar con las cuatro actividades guiadas para los estudiantes.

2. Componentes principales del kit

Antes de armar el circuito, es importante identificar cada pieza y su función dentro del sistema. Esta es la misma analogía con el cuerpo humano que se usa en la Actividad 1 para presentarla a los estudiantes.

Componente	Función en el sistema
Arduino Nano	El cerebro: recibe, almacena y ejecuta el programa creado por el estudiante. Traduce las líneas de código en señales eléctricas.
Driver L298N	El sistema nervioso / intermediario de potencia: recibe las señales del Arduino y usa la energía de la batería para mover los motores (el Arduino por si solo no puede manejar la corriente que necesitan).
Motores DC (x2)	Los músculos: generan el movimiento de las ruedas izquierda y derecha.
Batería portátil	La energía / el corazón: garantiza alimentación autónoma, sin depender del cable USB, para que el carrito se desplace libremente.
Chasis acrílico	La estructura física sobre la que se fijan todos los demás componentes.
LEDs (x3)	Retroalimentación visual: indican si el sistema está encendido o que movimiento está realizando el robot.
Buzzer	Retroalimentación auditiva: ayuda al estudiante a saber cuándo inicia o termina una secuencia programada.
Interruptor	Control físico de encendido y apagado; enseña la importancia de cerrar el circuito eléctrico.
Protoboard	Distribuye conexiones (por ejemplo, tierra común) entre varios componentes de forma ordenada.

3. Diagrama de conexiones (armado del circuito)

El siguiente diagrama muestra cómo se conecta cada componente. Sigue este orden antes de energizar el circuito por primera vez:

1. Fija todos los componentes sobre el chasis acrílico: batería e interruptor en la parte superior; Arduino Nano y L298N en el nivel intermedio; motores y ruedas en la base inferior.
2. Conecta la batería al interruptor, y el interruptor a la entrada de alimentación (+12V/GND) del L298N.
3. Conecta las salidas OUT1/OUT2 del L298N al motor izquierdo, y OUT3/OUT4 al motor derecho.
4. Conecta los pines digitales D8, D9, D10 y D11 del Arduino Nano a las entradas IN1, IN2, IN3 e IN4 del L298N respectivamente.
5. Conecta los LEDs a los pines D2, D3 y D4 del Arduino (cada uno con una resistencia de 220 ohm, cátodo a GND).
6. Conecta el buzzer al pin D5 del Arduino.
7. Asegúrate de unir el GND del Arduino con el GND del L298N (tierra común); usa la protoboard si necesitas más puntos de tierra.
8. Verifica visualmente la polaridad de cada conexión antes de encender el interruptor.

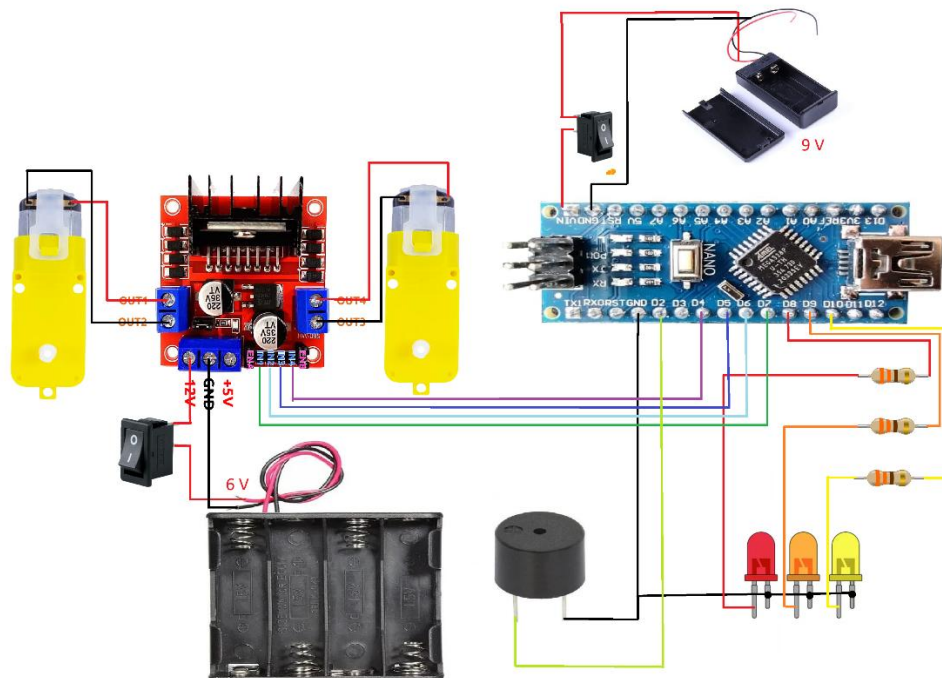


Diagrama de conexiones: Arduino Nano + Driver L298N + Motores DC + LEDs + Buzzer.

4. Instalación y configuración del software

Se recomienda usar dos herramientas complementarias: Arduino IDE para programar y cargar el código real al Arduino Nano, y Tinkercad para diseñar y simular el circuito de forma segura antes de armarlo físicamente (ideal para que los estudiantes prueben cambios sin riesgo de dañar componentes).

4.1 Arduino IDE

Instalación

1. Ingresa a la página oficial arduino.cc/en/software.
2. Descarga la versión correspondiente a tu sistema operativo (Windows, macOS o Linux).
3. Ejecuta el instalador y acepta la instalación de los drivers USB cuando el sistema lo solicite.
4. Conecta el Arduino Nano a la computadora mediante el cable USB.
5. Abre Arduino IDE y selecciona el menú Herramientas > Placa > Arduino Nano.
6. En Herramientas > Procesador, selecciona ATmega328P (Old Bootloader) si el Nano no carga con la opción por defecto (esto es común en clones de Arduino Nano).
7. En Herramientas > Puerto, selecciona el puerto COM (Windows) o /dev/tty (macOS/Linux) donde aparece el Arduino conectado.

Carga del programa

1. Copia el código de la sección 5 de este manual en un nuevo sketch de Arduino IDE.
2. Presiona el botón Verificar (icono de check) para compilar el código y revisar errores.
3. Presiona el botón Subir (icono de flecha) para cargar el programa al Arduino Nano.
4. Espera el mensaje "Carga completa" en la parte inferior de la ventana.
5. Desconecta el USB si el carrito ya tiene batería propia, cierra el interruptor y observa el movimiento.

4.2 Tinkercad (diseño y simulación)

Tinkercad Circuits permite armar el mismo circuito de forma virtual, simular el código y observar el comportamiento antes de tocar el hardware real. Es especialmente útil para que los estudiantes experimenten sin miedo a cometer errores.

Pasos básicos

1. Ingresa a tinkercad.com y crea una cuenta gratuita (o inicia sesión con una cuenta educativa).
2. En el menú, selecciona Circuits > Crear nuevo circuito.
3. Busca en el panel de componentes: Arduino Nano (o Uno, si Nano no está disponible en tu versión), driver de motor L298N (o modulo equivalente), motor DC, LED y buzzer, y arrástralos al área de trabajo.

4. Conecta los componentes en el simulador replicando el diagrama de la sección 3 (misma lógica de pines: D4-D8 al driver, D8-D10 a LEDs, D5 al buzzer).
5. Haz clic en el Arduino dentro del simulador y selecciona Editar código para pegar o escribir el programa en modo Texto (equivalente al lenguaje de Arduino IDE).
6. Presiona Iniciar simulación para observar el encendido de LEDs, el sonido del buzzer y el giro de los motores en tiempo real.
7. Modifica valores (por ejemplo, los tiempos de delay) y vuelve a simular para ver el efecto, tal como se plantea en la Actividad 3.

Sugerencia pedagógica

Antes de programar el Arduino físico, se recomienda que los estudiantes prueben primero sus cambios de código en Tinkercad. Esto reduce el riesgo de conexiones incorrectas y refuerza la comprensión del circuito de forma visual.

5. Código del programa y explicación detallada

A continuación se presenta el código completo que debe cargarse en el Arduino Nano, seguido de una explicación de cada sección.

```
// =====
// CARRITO ROBÓTICO EDUCATIVO - ESPOL
// Arduino Nano + L298N
// =====

// -----
// Pines del driver L298N
// -----
const int IN1 = 7;
const int IN2 = 6;
const int IN3 = 5;
const int IN4 = 4;

// -----
// LEDs indicadores
// -----
const int LED_ROJO = 8;
const int LED_AMARILLO = 9;
const int LED_VERDE = 10;

// -----
// Buzzer
// -----
const int BUZZER = 2;

// =====
```

```
// CONFIGURACIÓN INICIAL
// =====

void setup() {

    pinMode(IN1, OUTPUT);
    pinMode(IN2, OUTPUT);
    pinMode(IN3, OUTPUT);
    pinMode(IN4, OUTPUT);

    pinMode(LED_ROJO, OUTPUT);
    pinMode(LED_AMARILLO, OUTPUT);
    pinMode(LED_VERDE, OUTPUT);

    pinMode(BUZZER, OUTPUT);

    // Mantener los motores detenidos al iniciar
    detener();

    // Señal visual y sonora de inicio
    digitalWrite(LED_ROJO, HIGH);
    digitalWrite(LED_AMARILLO, HIGH);
    digitalWrite(LED_VERDE, HIGH);

    tone(BUZZER, 1000, 300);

    delay(300);

    digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_AMARILLO, LOW);
    digitalWrite(LED_VERDE, LOW);

    delay(700);
}

// =====
// FUNCIONES DE MOVIMIENTO
// =====

// Movimiento hacia adelante
void adelante() {

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}
```

```

// Movimiento hacia atrás
void atras() {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

// Detener motores
void detener() {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}

// Giro hacia la derecha
void derecha() {

    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);
}

// Giro hacia la izquierda
void izquierda() {

    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(IN3, HIGH);
    digitalWrite(IN4, LOW);
}

// =====
// SECUENCIA PRINCIPAL
// =====

```

```

void loop() {

    // -----
    // 1. Avanzar durante 3 segundos
    // -----
    digitalWrite(LED_ROJO, HIGH);

    adelante();

    delay(3000);

    // -----
    // 2. Giro a la derecha
    // -----
    digitalWrite(LED_ROJO, LOW);
    digitalWrite(LED_AMARILLO, HIGH);

    tone(BUZZER, 1500, 150);

    derecha();

    delay(900);

    // -----
    // 3. Avanzar durante 2.5 segundos
    // -----
    digitalWrite(LED_AMARILLO, LOW);
    digitalWrite(LED_VERDE, HIGH);

    adelante();

    delay(2500);

    // -----
    // 4. Giro a la izquierda
    // -----
    digitalWrite(LED_VERDE, LOW);
    digitalWrite(LED_ROJO, HIGH);

    tone(BUZZER, 1500, 150);

    izquierda();

    delay(900);
}

```

```
// -----
// 5. Detener el carrito
// -----
detener();

digitalWrite(LED_ROJO, LOW);

delay(300);
}
```

5.1 Explicación por secciones

Declaración de pines

Al inicio del código se declaran constantes que asignan un nombre legible a cada pin físico del Arduino: IN1 a IN4 (pines 8 a 11) controlan el driver L298N; LED1, LED2 y LED3 (pines 2, 3 y 4) controlan los LEDs; y BUZZER (pin 5) controla el zumbador. Usar constantes en lugar de números sueltos hace el código más fácil de leer y de modificar.

void setup()

Esta función se ejecuta una sola vez al encender o reiniciar el Arduino. Aquí se define, con pinMode(), que cada pin usado será una salida (OUTPUT), es decir, que el Arduino enviara señales eléctricas hacia esos pines en lugar de leerlas. También se llama a detener() para asegurar que los motores inicien apagados, se encienden los tres LEDs y se hace sonar el buzzer brevemente (300 ms) como señal de que el sistema arranco correctamente, y finalmente se espera 1 segundo (delay(1000)) antes de iniciar la secuencia de movimiento.

Funciones de movimiento

Cada función (adelante, atrás, detener, derecha, izquierda) controla el sentido de giro de los motores mediante combinaciones de HIGH y LOW en los pines IN1-IN4, que llegan al L298N:

- adelante(): IN1 y IN3 en HIGH, IN2 e IN4 en LOW. Ambos motores giran en el mismo sentido hacia adelante.
- atras(): se invierten las señales (IN1 e IN3 en LOW, IN2 e IN4 en HIGH), lo que invierte la polaridad en los motores y los hace girar en reversa.
- detener(): los cuatro pines en LOW; el L298N no entrega corriente a los motores y estos se frenan.
- derecha() / izquierda(): se activan combinaciones distintas en cada lado (un motor gira hacia adelante y el otro hacia atrás), logrando que el carrito gire sobre su propio eje.

Este es el principio físico clave que se explica en la Actividad 2: cada instrucción de código produce una acción física inmediata y observable en el carrito.

void loop()

Esta función se repite indefinidamente mientras el Arduino este encendido. Define la secuencia de movimientos del carrito, combinando las funciones anteriores con delay() (pausas en milisegundos) y encendiendo un LED distinto en cada tramo para dar retroalimentación visual de en qué parte de la secuencia se encuentra el robot:

- Avanza 3 segundos (LED1 encendido).
- Gira a la derecha 0.9 segundos (LED2 encendido).
- Avanza 2.5 segundos (LED3 encendido).
- Gira a la izquierda 0.9 segundos (LED1 encendido nuevamente).
- Se detiene 300 milisegundos con los tres LEDs apagados, y el ciclo vuelve a empezar.

Modificar los valores de `delay()` es exactamente lo que se pide a los estudiantes en la Actividad 3 (Modificando el Recorrido): al aumentar, por ejemplo, `delay(3000)` a `delay(5000)` en la instrucción `adelante()`, el carrito recorrerá una distancia física mayor, permitiendo descubrir la relación entre tiempo de instrucción y distancia recorrida.

6. Guía paso a paso de las actividades con estudiantes

Esta sección detalla cada actividad planteada en las diapositivas, indicando el objetivo, la dinámica y los pasos concretos a seguir con los estudiantes.

Actividad 1: Conociendo mi Robot

Objetivo: identificar los principales componentes del carrito y reconocer para que sirve cada uno antes de empezar a programar.

Pasos

1. Presenta el kit desarmado o armado a los estudiantes.
2. Pide que observen cada pieza y relaciona cada una con una analogía del cuerpo humano: Arduino = Cerebro, L298N = Sistema nervioso, Motores = Músculos, Batería = Energía / Corazón.
3. Usa la tabla de componentes (sección 2) como apoyo visual para reforzar cada analogía.
4. Verifica que cada estudiante pueda nombrar al menos tres componentes y su función antes de continuar.

Actividad 2: Mi Primer Movimiento

Objetivo: comprender que cada palabra escrita en el programa produce una acción física en el mundo real (instrucción = acción).

Pasos

1. Con el código de la sección 5 ya cargado, enciende el carrito desde el interruptor.
2. Ejecuta el programa básico proporcionado (el mismo de la sección 5).
3. Pide a los estudiantes que observen como el carrito avanza, gira y se detiene.
4. Muestra el código en la pantalla y relaciona cada movimiento visto con la función correspondiente (`adelante()`, `derecha()`, `izquierda()`, `detener()`).

Actividad 3: Modificando el Recorrido

Objetivo: introducir la modificación del programa, cambiando los valores numéricos de tiempo en el código (por ejemplo, `delay(1000)` a `delay(3000)`), para descubrir el principio de prueba y error.

Pasos

1. Abre el código en Arduino IDE (o en Tinkercad si se trabaja en simulación).
2. Identifica junto con el estudiante los valores de `delay()` dentro de la función `loop()`.
3. Cambia uno de los valores, por ejemplo, el `delay(3000)` de la primera llamada a `adelante()`, a un valor mayor o menor.
4. Guarda, sube el código nuevamente (o vuelve a simular en Tinkercad) y observa el resultado.
5. Guía la reflexión: si aumenta el tiempo de la instrucción `adelante()`, el carrito recorrerá una distancia física mayor. Repite el ciclo de prueba y error con distintos valores.

Actividad 4: Programa Tu Recorrido

Objetivo: el estudiante debe programar una secuencia lógica para llevar el carrito desde un punto inicial (A) hasta un punto objetivo (B), desarrollando planificación, dirección y resolución de problemas.

Pasos

1. Define físicamente un punto de partida (A) y un punto objetivo (B) en el suelo, con algún obstáculo intermedio si se desea aumentar la dificultad.
2. Planificación: pide al estudiante decidir cuanto tiempo debe avanzar en línea recta el carrito para no pasarse del objetivo (ajustando los `delay()` de `adelante()`).
3. Dirección: pide analizar espacialmente hacia que dirección girar (derecha o izquierda) para esquivar el obstáculo, y ajustar las llamadas a `derecha()` o `izquierda()` según corresponda.
4. Resolución: pide programar el momento exacto para frenar (llamada a `detener()`), llegando lo más cerca posible del punto B.
5. Carga el código final, ejecuta y evalúa junto con el estudiante que tan cerca llegó del objetivo, repitiendo el ciclo de ajuste si es necesario.

7. Pruebas y ajustes antes de usar el kit

Antes de utilizar los kits con los estudiantes, se recomienda realizar el siguiente control de calidad:

- Verificación de las conexiones de motores y polaridad correcta.
- Comprobación visual de los LEDs e intensidad del buzzer.

- Pruebas repetidas del interruptor y estado de las baterías.
- Ejecución de un código de diagnóstico simple (por ejemplo, encender cada LED y motor por separado) para identificar errores antes de la sesión.
- Ajustes físicos del chasis para garantizar un desplazamiento recto y sin fricción.

8. Resultados de aprendizaje esperados

Al finalizar las actividades, se espera que los estudiantes puedan:

- Reconocer componentes electrónicos básicos y comprender la función de un microcontrolador (Electrónica básica).
- Relacionar la abstracción del código de programación con movimientos tangibles (Programación).
- Desarrollar el pensamiento lógico para crear y modificar secuencias paso a paso (Lógica computacional).
- Aprender a resolver problemas y corregir errores mediante el método de prueba y error (Resolución de problemas).

Mucho más que un juguete

El objetivo central no es que los niños simplemente vean un robot moviéndose de manera autónoma. La misión de este manual es convertirlos en creadores: que puedan comprender, modificar y programar el carrito ellos mismos.